

be displayed in the mini buffer.

You can also have nested sexps, wherein the innermost expressions are evaluated first.

```
(+ 1 (* 2 3))
```

Similar to the '+' operation, you can use pre-defined keywords or functions in Emacs Lisp programs. The *format* function is similar to your *printf* statement in the C programming language.

```
(format "Hello, World!")
```

You can store a value in a variable using the *setq* operation as shown below:

```
(setq IST "Indian Standard Time")
```

```
IST      ; No enclosing parenthesis!  
          ; produces "Indian Standard Time"
```

You can find the data type, using the *type-of* function. A few examples are shown below:

```
(type-of 1)           ; produces integer  
(type-of 3.1415)      ; produces float  
(type-of "A string")  ; produces string  
(type-of :foo)        ; produces symbol  
(type-of t)           ; produces symbol  
(type-of '(1 2))      ; produces cons  
(type-of [1 "two" :three]) ; produces vector
```

In the Bash shell, you can escape evaluation by using the backslash. Similarly, you can prevent the Emacs Lisp interpreter from evaluating an expression using quotes. For example:

```
(+ 1 2)      ; produces 3  
'(+ 1 2)    ; produces the list (+ 1 2)
```

progn

The *progn* statement is used to execute several sexps. For example:

```
(progn  
  (setq title "Introduction to Emacs Lisp")  
  (setq author "Robert J. Chassell")  
  (format "%s by %s" title author)) ; produces "Introduction  
to Emacs Lisp by Robert J. Chassell"
```

Functions

You can define your own function using the *defun* built-in keyword. The 'say' function simply prints the string "Hello,

World!" in the following example:

```
(defun say ()  
  (format "Hello, World!"))
```

In order to execute the function, invoke it inside parenthesis, as shown below:

```
(say)      ; produces "Hello, World!"
```

Arguments can be passed to a function. The 'square' function is demonstrated below:

```
(defun square (x)  
  (* x x))  
(square 3) ; produces 9
```

You can also store a list of names and retrieve them using the *car* and *cdr* functions as illustrated below:

```
(setq teams '("GL" "DD" "KKR" "MI" "SRH" "RPS" "RCB" "KXIP"))  
  
(car teams)      ; produces "GL"  
  
(cdr teams)      ; produces "DD" "KKR" "MI" "SRH" "RPS"  
"RCB" "KXIP"  
  
(car (cdr teams)) ; produces "DD"
```

let

The *let* statement allows you to bind a value to a variable in a local context. GNU Emacs 24 has lexical binding support. An example is given below:

```
(defun hello (name)  
  (let ((new-name (concat "Hello " name)))  
    new-name  
  ))  
  
(hello "Mark") ; produces "Hello Mark"
```

Conditions

Conditions can be checked using the *if* and *cond* statements. The *if* statement takes a condition, and if the condition evaluates to true, the sexp immediately following the condition is executed. Otherwise, the else-form is evaluated. The syntax is:

```
(if condition then-form else-form)
```

Here's an example:

```
(if t  
  (format "True"))
```